ANNDL CIE 1 - Quiz Solutions (Part 2)

Subject: Artificial Neural Networks and Deep Learning **Questions:** 3 and 4

1. Difference between BPTT and Regular Backpropagation:

- **Regular Backpropagation (Feed-Forward Networks):** In a standard Feed-Forward Neural Network (FNN), the information flows in one direction (input to output). Backpropagation computes gradients by traversing from the output layer back to the input layer through a fixed, static number of layers.
- **Backpropagation Through Time (BPTT) (RNNs):** Recurrent Neural Networks (RNNs) process sequential data and have cyclic connections. To train them, we cannot use standard backprop directly because the output at time t depends on the state at $t-1$.
  - **Unrolling:** BPTT operates by "unfolding" or "unrolling" the RNN over time. If the input sequence has length T, the RNN is conceptually converted into a Feed-Forward network with T layers.
  - **Weight Sharing:** Unlike a standard deep network where each layer has unique weights, in the unrolled RNN, the weight matrices (Input-to-Hidden U, Hidden-to-Hidden W, Hidden-to-Output V) are **shared (identical)** across all T time steps.
  - **Gradient Accumulation:** When calculating the gradient for a weight (e.g., W), BPTT sums the gradients calculated at each time step.

2. Challenges in RNNs (Vanishing and Exploding Gradients):

The core issue arises from the chain rule applied over many time steps. To calculate the gradient of the error at time T with respect to the input at time 1, we essentially multiply the recurrent weight matrix W by itself T times (along with the derivative of the activation function).

- **Gradient Formula Approximation:** $\frac{\partial \mathcal{L}}{\partial W} \propto \prod_{t=1}^{T} W \cdot f'(h_t)$
- **Vanishing Gradients:**
  - **Cause:** If the largest eigenvalue (spectral radius) of the weight matrix W is less than 1 (or if the activation function derivative, like sigmoid/tanh, is < 1), the gradients decay exponentially as they are propagated back through time (e.g., $0.9^{100} \approx 0$).
  - **Effect:** The weights are updated only based on near-term effects. The network **fails to learn long-term dependencies** (e.g., remembering a word from the beginning of a paragraph to predict the end).
- **Exploding Gradients:**
  - **Cause:** If the spectral radius of W is greater than 1, the gradients grow exponentially (e.g., $1.1^{100} \approx 13780$).
  - **Effect:** The weight updates become massively large, causing the network weights to oscillate wildly or diverge to `NaN` (Not a Number), leading to complete training failure.

5 b) CNN Approaches for Video Classification

Question: Explain any two approaches of CNN for video classification?

Solution:

Video classification requires analyzing both spatial information (what objects are in the scene) and temporal information (how they move over time).

Approach 1: 3D Convolutional Networks (3D CNNs / C3D)

- **Concept:** Standard CNNs use 2D kernels ($width \times height$) that slide over an image. 3D CNNs use 3D kernels ($width \times height \times time$) that slide over a "volume" of video frames.
- **Mechanism:** The filter convolves across the spatial dimensions and the temporal dimension simultaneously.
- **Advantage:** This allows the network to extract motion features (like a hand waving) directly from the raw pixel volume, preserving temporal relationships in the early layers.
- **Output:** The output of a 3D convolution is another 3D volume, preserving temporal depth until the final fully connected layers.

Approach 2: Two-Stream Networks

- **Concept:** This architecture explicitly separates the video into two distinct components: appearance and motion. It uses two parallel 2D CNNs.
- **Stream 1 (Spatial Stream):** Takes a single frame (RGB image) as input. Its job is to recognize objects and scenes (e.g., "This is a baseball field", "This is a bat").
- **Stream 2 (Temporal Stream):** Takes a stack of **Optical Flow** frames as input. Optical flow represents the displacement of pixels between frames. Its job is to recognize the movement dynamics (e.g., "swinging motion") regardless of what the object looks like.
- **Fusion:** The predictions (class scores) from both streams are fused at the end (e.g., by averaging or using an SVM) to get the final classification. This often yields higher accuracy than 3D CNNs alone.

6) RNN vs. LSTM vs. GRU

Question: Compare the performance of basic RNNs, LSTMs, and GRUs in handling long-term dependencies in time-series forecasting. What are the strengths and weaknesses of each model?

Solution:

1. Basic RNN (Recurrent Neural Network)

- **Handling Long-term Dependencies: Poor.** Due to the vanishing gradient problem (explained in 5a), basic RNNs cannot retain information over long sequences. They typically only look back 5-10 time steps effectively.

- **Strengths:** Simple architecture; fast to train (fewer parameters); computationally inexpensive.
- **Weaknesses:** Fails catastrophically at learning long-term patterns; prone to gradient explosion.

2. LSTM (Long Short-Term Memory)

- **Handling Long-term Dependencies: Excellent.** LSTMs introduce a "memory cell" (C_t) distinct from the hidden state. They use a gating mechanism (Input, Forget, and Output gates) to regulate information flow. The **Forget Gate** allows the network to keep a memory for thousands of time steps without it degrading, effectively creating a "constant error carousel" that preserves gradients.
- **Strengths:** Can bridge very large time lags (e.g., hundreds of steps); standard choice for complex sequence tasks.
- **Weaknesses:** Complex structure (4 interacting layers per cell); many parameters to train; slower training and inference time; prone to overfitting on small datasets.

3. GRU (Gated Recurrent Unit)

- **Handling Long-term Dependencies: Very Good.** GRUs are a simplified variation of LSTMs. They merge the cell state and hidden state into one and combine the Forget and Input gates into a single **Update Gate**. They perform comparably to LSTMs on most long-term dependency tasks.
- **Strengths:** Simpler than LSTM (2 gates vs. 3); fewer parameters; often trains faster than LSTM; efficient for smaller datasets.
- **Weaknesses:** Theoretically slightly less expressive than LSTMs (since they lack the separate output gate and memory cell), though in practice, the difference is often negligible.

Summary Table:

Feature	Basic RNN	LSTM	GRU
Long-term Memory	Very Poor	Excellent	Excellent
Complexity	Low	High	Medium
Training Speed	Fast	Slow	Medium-Fast
Vanishing Gradient	Susceptible	Robust	Robust